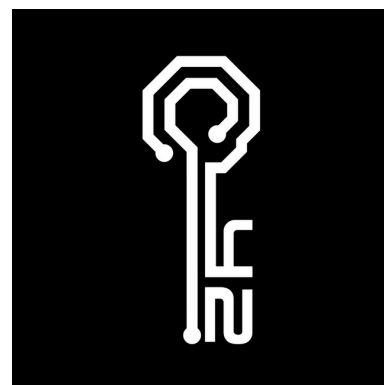


Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра № 42 «Криптология и кибербезопасность»



ОТЧЕТ

**О выполнении лабораторной работы № 2
«Введение в параллельные вычисления. Технология OpenMP»**

Студент: Доценко В. А.

Группа: Б23-505

Преподаватель: Куприяшин М.А.

Москва — 2025

Оглавление

1. Цель работы	3
2. Рабочая среда	3
3. Анализ последовательного алгоритма	3
3.1. Описание	3
3.2. Оценка	3
3.3. Блок-схема	4
4. Анализ параллельного алгоритма	5
4.1. Описание	5
4.2. Оценка параметров	5
4.3. Директивы OpenMP	5
4.4. Блок-схемы алгоритмов	7
5. Результаты экспериментов	9
6. Заключение	11
7. Приложение	13
7.1. Программные коды	13
7.2. Таблицы	18

1. Цель работы

Приобрести навыки разработки параллельной программы путём обнаружения ресурса параллелизма в имеющейся последовательной реализации.

2. Рабочая среда

Модель процессора: Apple M3.

Количество ядер / потоков: 8 / 8.

Объем оперативной памяти: 24 ГБ.

Разрядность и версия ОС: macOS Sequoia 15.6.

Среда разработки: VS Code, clang 16.0.0.

Версия OpenMP: 4.0 (`_OPENMP = 202011`).

3. Анализ последовательного алгоритма

3.1. Описание

1. Создаётся массив из N элементов.
2. Массив заполняется случайными числами с использованием функции `rand()`.
3. Осуществляется последовательный проход по массиву пока не будет найден искомый элемент.
4. Индекс ячейки массива, содержащей искомый элемент, выводится на экран.

3.2. Оценка

Худшая – полный проход $O(n)$.

Лучшая – первый элемент – искомый $O(1)$.

3.3. Блок-схема

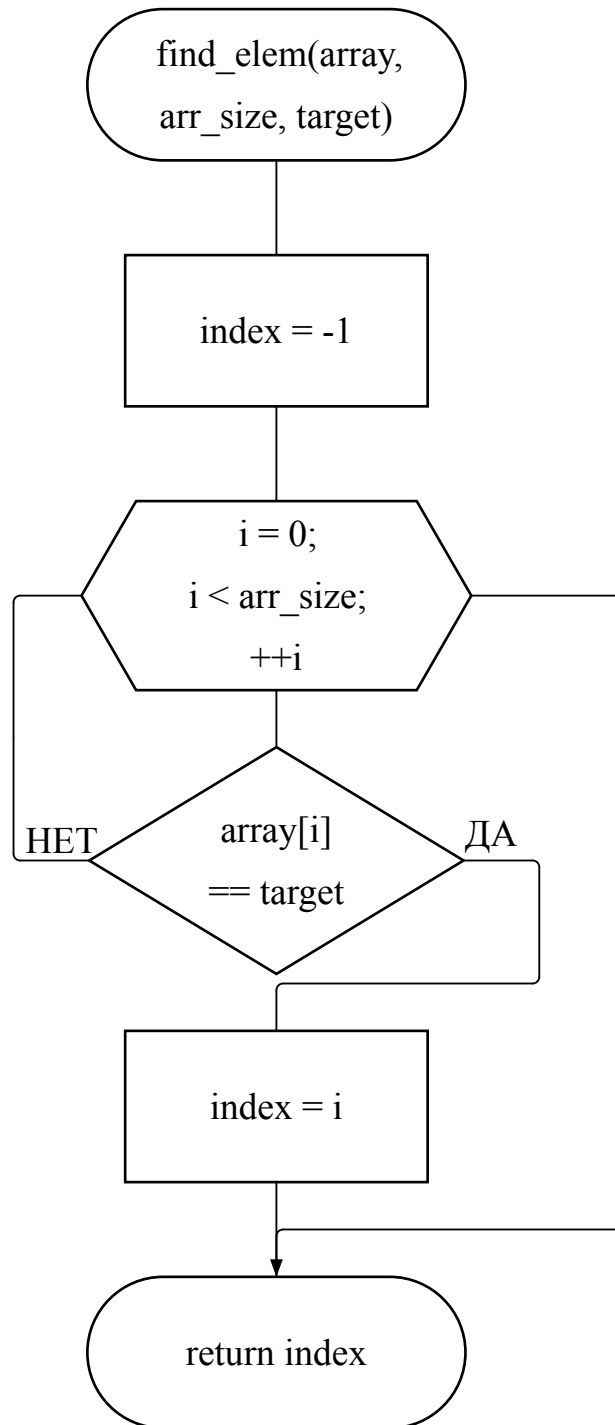


Рисунок 1 – Блок-схема алгоритма, описанного в п. 3.1

4. Анализ параллельного алгоритма

4.1. Описание

Для этого алгоритма я выявил три возможных способа распараллеливания:

1. **Использовать `#pragma omp cancel for` + `#pragma omp critical`.** После того, как индекс найден, происходит отмена оставшихся итераций цикла, потоки мгновенно прекращают свою работу. Теоретически, скорость выполнения этого алгоритма тем выше, чем раньше будет найден подходящий элемент.
2. **Синхронизация через `atomic`.** этот вариант сильно снижает накладные расходы по сравнению с `critical` и `cancel for`, однако потоки будут продолжать свою работу даже после того, как элемент будет найден, будут пропускаться циклы.
3. **Разделение на блоки + редукция минимума индекса.** Полный обход массива с редукцией теоретически должен быть менее эффективным, так как требует больше итераций цикла. Однако, накладные расходы на остановку других потоков могут невелировать полученное преимущество.

4.2. Оценка параметров

Ускорение: $S_p = \frac{T_1}{T_p}$, где T_1 – время работы в линейном режиме (1 поток), T_p – время работы при p потоках.

Эффективность: $E_p = \frac{S_p}{p}$, где S_p - ускорение, p – количество задействованных потоков.

4.3. Директивы OpenMP

`#pragma omp for`

Директива для распараллеливания цикла `for`. По умолчанию применяется `schedule(static)`, то есть нагрузка распределится равномерно между потоками.

`#pragma omp parallel num_threads(threads) default(none) shared(arr, N, target, index, found)`

Директива, определяющая регион общего доступа к `array`, размера `N`, для поиска (совместное чтение) `target`. При добавлении `reduction(min:index)` произойдет редукция индекса к минимальному.

#pragma omp critical

Директива, обозначающая критический регион, доступ к которому в каждый момент времени имеет только 1 поток.

#pragma omp cancel for и #pragma omp cancellation point for

Директивы остановки выполнения цикла for в регионе для всех потоков и установки точки для проверки запроса на остановку.

#pragma omp atomic read

Атомарное чтение гарантирует, что поток получит актуальные данные, дешевле, чем чтение через critical.

#pragma omp atomic capture

атомарная комбинация чтения и записи (или другой пары операций) с возможностью «захватить» предыдущее или текущее значение в переменную

4.4. Блок-схемы алгоритмов

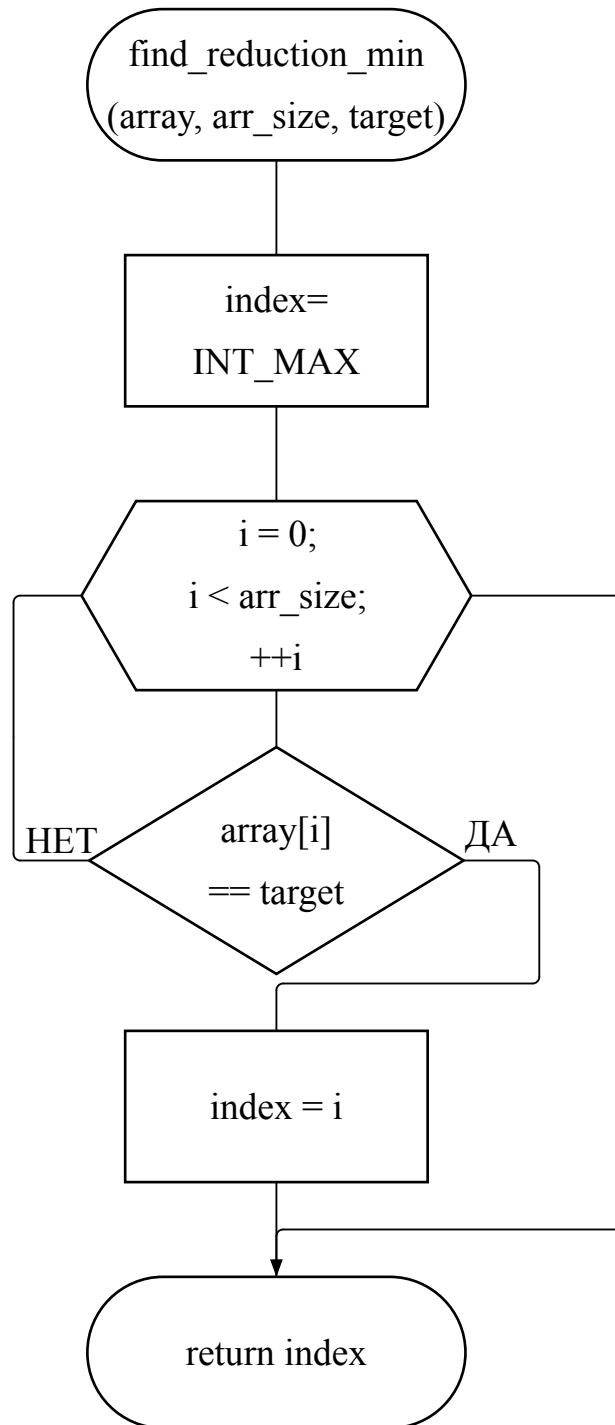


Рисунок 2 – Блок-схема параллельного алгоритма с отменой циклов (через `atomic` или `cancel for`)

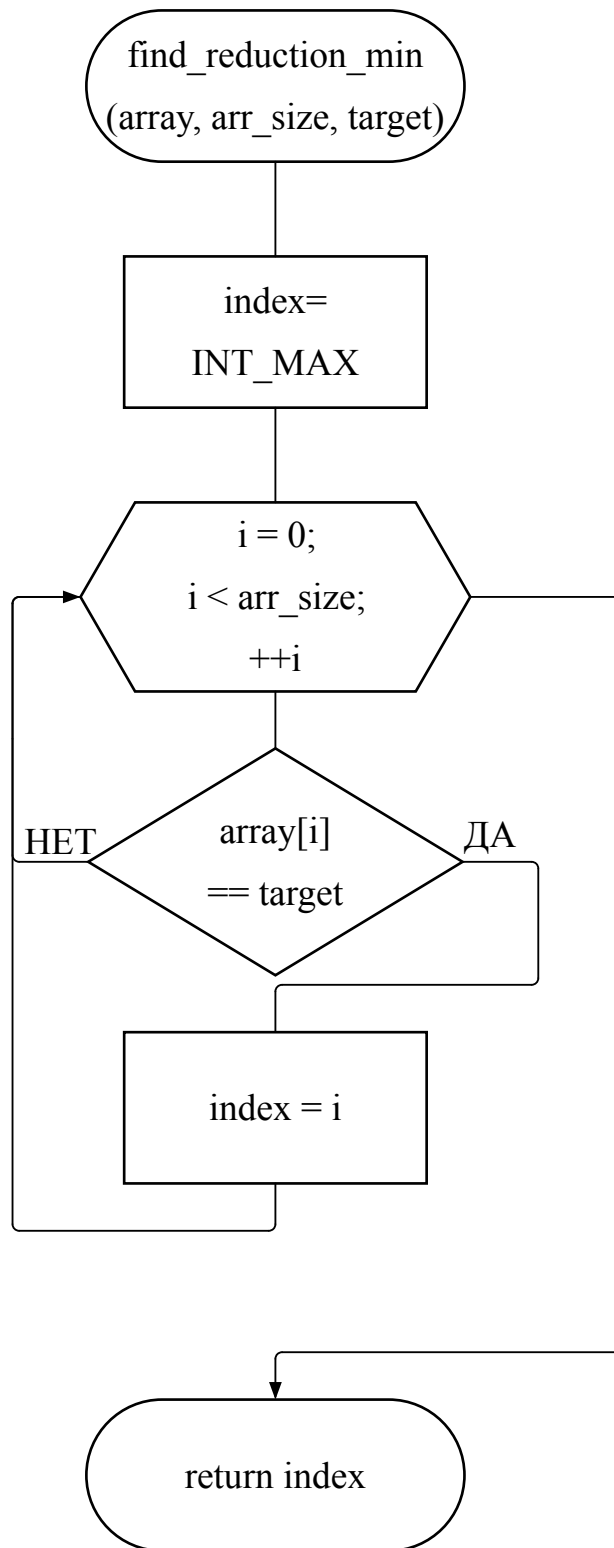


Рисунок 3 – Блок-схема параллельного алгоритма с применением редукции

5. Результаты экспериментов

Каждая точка на графике была получена путем повторения эксперимента 20 раз, на одинаковых исходных данных для каждого повтора. Для всех экспериментов количество элементов в массиве было равно 30 миллионам, элементы генерировались случайно при помощи функции `rand()`, `target` так же генерировался при помощи `rand()`;

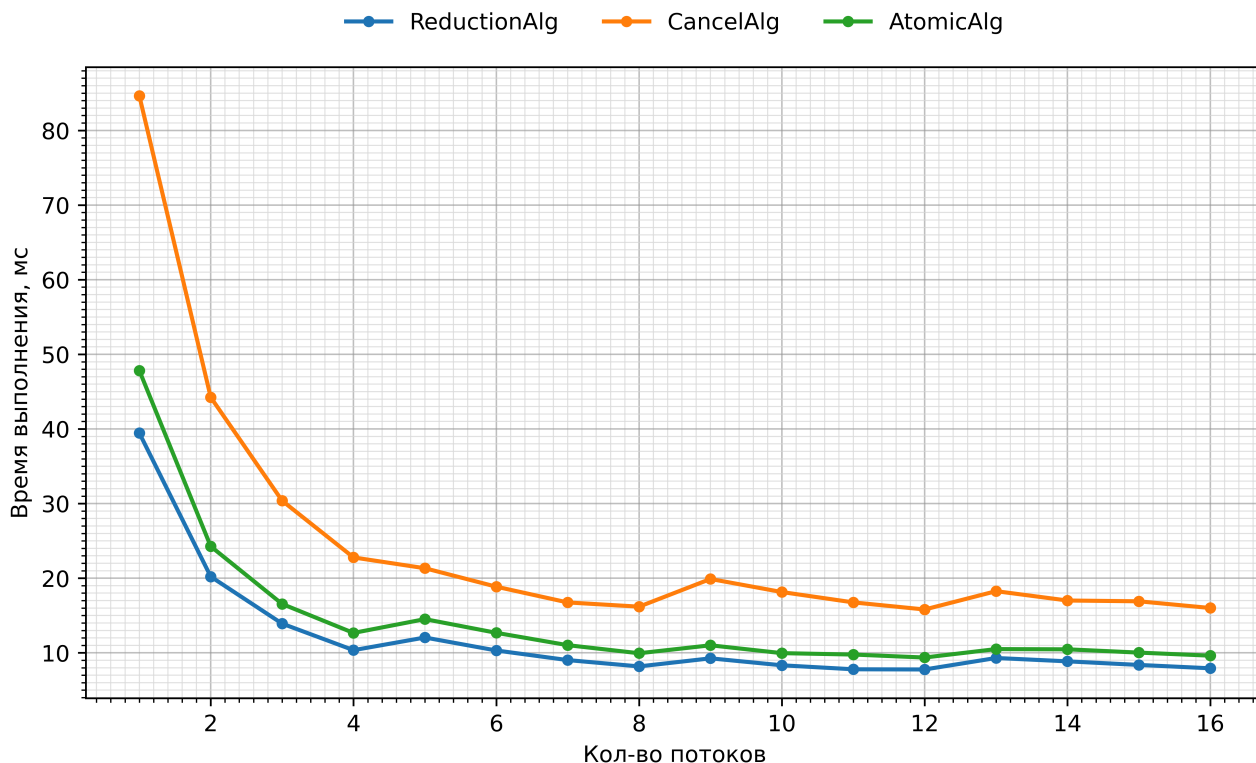


Рисунок 4 – График зависимости времени выполнения параллельных алгоритмов от количества потоков

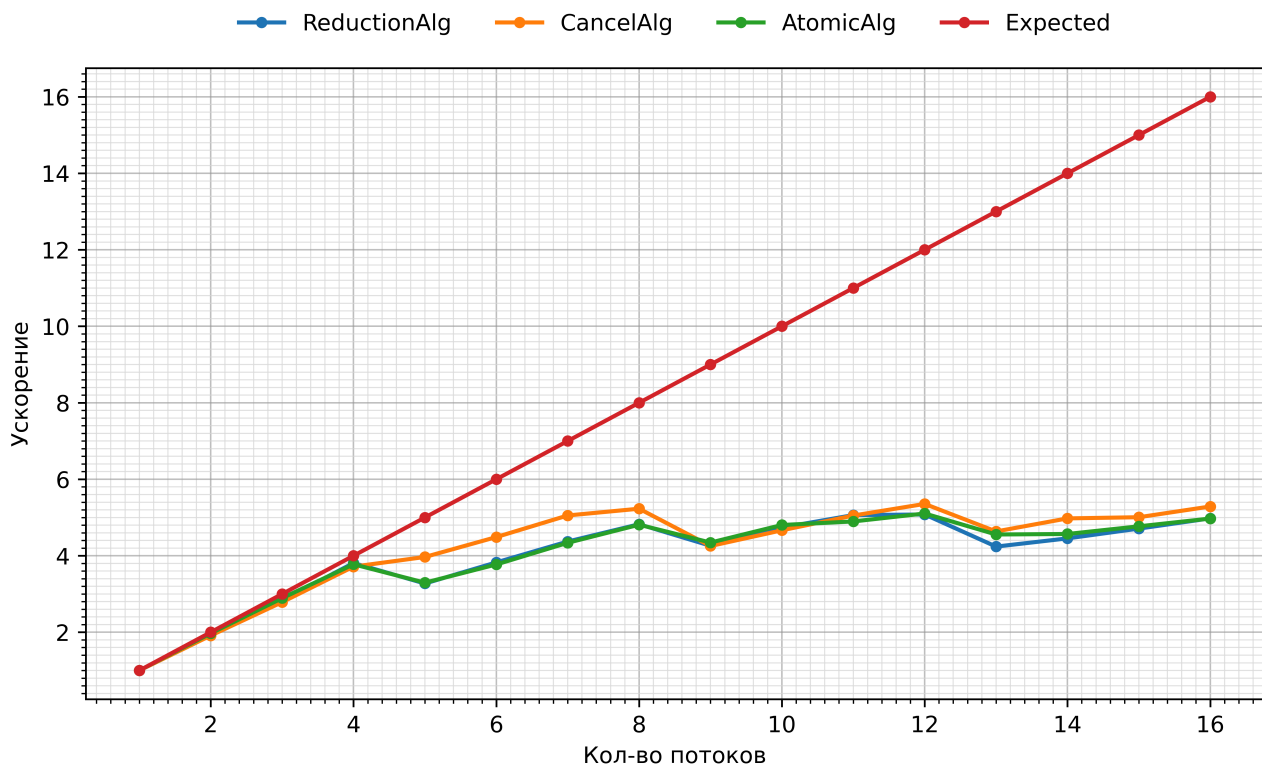


Рисунок 5 – График зависимости ускорения параллельных алгоритмов от количества потоков

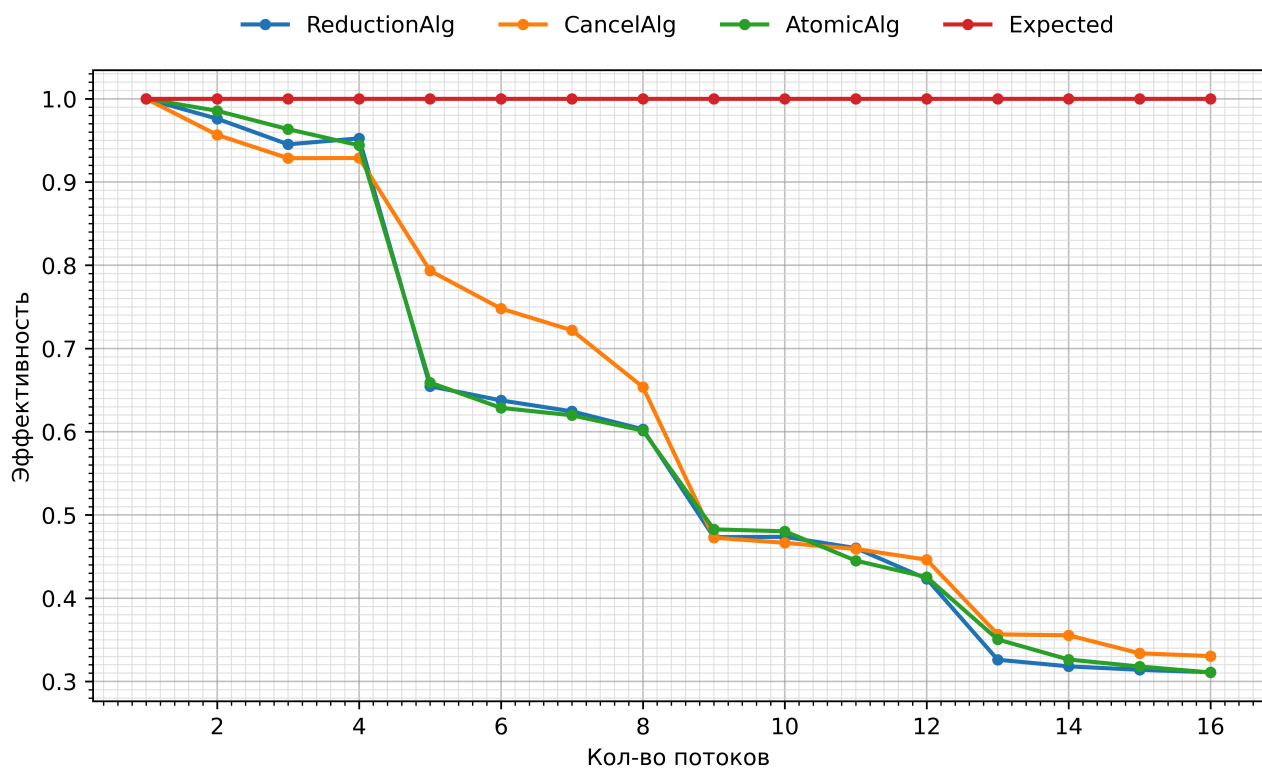


Рисунок 6 – График зависимости эффективности параллельных алгоритмов от количества потоков

6. Заключение

В ходе выполнения лабораторной работы были приобретены практические навыки выделения ресурса параллелизма в последовательном алгоритме и его реализации с использованием технологии OpenMP.

Был рассмотрен последовательный алгоритм поиска элемента в массиве, произведена оценка сложности: в лучшем случае – константная, в худшем – линейная. Построена блок-схема (Рисунок 1).

Выявлена область программы, где возможно применение параллельных вычислений, и реализованы 3 варианта алгоритма поиска:

1. Проход по всей длине массива с применением редукции
2. Остановка при успешном сравнении
3. Пропуск циклов при успешном сравнении и синхронизация через `atomic`

Построены блок-схемы этих алгоритмов (Рисунок 2 и Рисунок 3).

Экспериментально были получены значения времени работы, ускорения и эффективности для каждого из алгоритмов (Таблица 1). Самой быстрой реализацией оказалась реализацией оказалась `reduction`-алгоритм. Незначительно отстаёт по скорости `atomic`-алгоритм, а самый медленный – `cancel-for` алгоритм. Наиболее большая разница в скорости между `cancel` и остальными наблюдается на 2-4 потоках. Это связано с накладными расходами на вызов `#pragma omp cancel for` и более медленной синхронизацией через `critical`.

На графиках (Рисунок 4, Рисунок 5, Рисунок 6), как и в прошлой лабораторной, наблюдается снижение эффективности всех видов распараллеливания алгоритмов после достижения 4, 8 и 12 потоков. Это связано с устройством процессоров Apple M-серии (подробнее см. заключение лабораторной работы №1). Однако, снижение эффективности при пяти потоках у алгоритма через отмену циклов значительно меньше, чем у других алгоритмов. Это означает, что `cancel`-алгоритм даёт большее повышение производительности при росте количества потоков, чем другие алгоритмы. Кроме того, производительность этого алгоритма тем выше, чем раньше будет найден подходящий элемент.

Таким образом, в работе достигнута цель построения параллельного алгоритма из последовательного с последующим анализом эффективности.

7. Приложение

7.1. Программные коды

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>
#include <limits.h>

#include <omp.h>
#include <limits.h>

int find_cancel(int *arr, int size, int target, int threads) {
    int index = -1;

    #pragma omp parallel num_threads(threads) default(none) \
        shared(arr, size, target, index)
    {
        #pragma omp for
        for (int i = 0; i < size; ++i) {
            if (arr[i] == target) {
                #pragma omp critical
                {
                    if (index == -1)
                        index = i;
                }
                #pragma omp cancel for
            }

            #pragma omp cancellation point for
        }
    }
    return index;
}

int find_atomic(int *arr, int size, int target, int threads) {
    int index = -1;

    #pragma omp parallel num_threads(threads) default(none) \
        shared(arr, size, target, index)
    {
        #pragma omp for
        for (int i = 0; i < size; ++i) {
            int current;
            #pragma omp atomic read
            current = index;
            if (current != -1) {
                continue;
            }

            if (arr[i] == target) {
                int prev;
                #pragma omp atomic capture
                {
```

```

        prev = index;
        index = (prev == -1) ? i : prev;
    }
}
}

return index;
}

int find_reduction_min(int *arr, int size, int target, int threads) {
    int index = INT_MAX;

    #pragma omp parallel for num_threads(threads) \
                        default(none) \
                        shared(arr, size, target) \
                        reduction(min:index)
    for (int i = 0; i < size; ++i) {
        if (arr[i] == target) {
            if (i < index) index = i;
        }
    }

    return (index == INT_MAX) ? -1 : index;
}

void getExperimentDots(
    int* array,
    int sz,
    int target,
    int threads,
    int repeats,
    FILE* file_res,
    FILE* file_speedup,
    FILE* file_eff,
    int* Tlcancel,
    int* Tlatomic,
    int* Tlreduction)
{
    double timeCancel = 0;
    double timeAtomic = 0;
    double timeReduction = 0;
    double start, end;
    for (int i = 0; i < repeats; ++i) {
        start = omp_get_wtime();
        find_cancel(array, sz, target, threads);
        end = omp_get_wtime();
        timeCancel += end - start;
    }
    timeCancel /= 10;
    timeCancel *= 1000;
    if (threads == 1) {
        *Tlcancel = timeCancel;
    }

    for (int i = 0; i < repeats; ++i) {
        start = omp_get_wtime();

```

```

        find_atomic(array, sz, target, threads);
        end = omp_get_wtime();
        timeAtomic += end - start;
    }
    timeAtomic /= 10;
    timeAtomic *= 1000;
    if (threads == 1) {
        *Tlatomic = timeAtomic;
    }

    for (int i = 0; i < repeats; ++i) {
        start = omp_get_wtime();
        find_reduction_min(array, sz, target, threads);
        end = omp_get_wtime();
        timeReduction += end - start;
    }
    timeReduction /= 10;
    timeReduction *= 1000;
    if (threads == 1) {
        *Tlreduction = timeReduction;
    }
    double realSpeedupCancel = (*Tlcancel) / timeCancel;
    double realSpeedupAtomic = (*Tlatomic) / timeAtomic;
    double realSpeedupReduction = (*Tlreduction) / timeReduction;
    if (file_res != NULL) {
        fprintf(file_res, "%d %lf %lf %lf\n", threads, timeReduction, timeCancel, timeAtomic);
    }
    if (file_speedup != NULL) {
        fprintf(file_speedup, "%d %lf %lf %lf %d\n", threads, realSpeedupReduction, realSpeedupCancel,
realSpeedupAtomic, threads);
    }
    if (file_eff != NULL) {
        fprintf(file_eff, "%d %lf %lf %lf %d\n", threads, realSpeedupReduction / threads,
realSpeedupCancel / threads, realSpeedupAtomic / threads, 1);
    }
}

int main(int argc, char** argv)
{
    srand(time(NULL));

    FILE* file_res = fopen("results/runtime", "w");
    fprintf(file_res, "3\n");
    fprintf(file_res, "ReductionAlg\n");
    fprintf(file_res, "CancelAlg\n");
    fprintf(file_res, "AtomicAlg\n");
    fprintf(file_res, "Кол-во потоков\n");
    fprintf(file_res, "Время выполнения, мс\n");
    fprintf(file_res, "none\n");

    FILE* file_speedup = fopen("results/speedup", "w");
    fprintf(file_speedup, "4\n");
    fprintf(file_speedup, "ReductionAlg\n");
    fprintf(file_speedup, "CancelAlg\n");
    fprintf(file_speedup, "AtomicAlg\n");
    fprintf(file_speedup, "Expected\n");
    fprintf(file_speedup, "Кол-во потоков\n");

```

```

fprintf(file_speedup, "Ускорение\n");
fprintf(file_speedup, "none\n");

FILE* file_eff = fopen("results/efficiency", "w");
fprintf(file_eff, "4\n");
fprintf(file_eff, "ReductionAlg\n");
fprintf(file_eff, "CancelAlg\n");
fprintf(file_eff, "AtomicAlg\n");
fprintf(file_eff, "Expected\n");
fprintf(file_eff, "Кол-во потоков\n");
fprintf(file_eff, "Эффективность\n");
fprintf(file_eff, "none\n");

FILE* file_det_eff_t = fopen("results/det_efficiency_t", "w");
fprintf(file_det_eff_t, "4\n");
fprintf(file_det_eff_t, "ReductionAlg\n");
fprintf(file_det_eff_t, "CancelAlg\n");
fprintf(file_det_eff_t, "AtomicAlg\n");
fprintf(file_det_eff_t, "Expected\n");
fprintf(file_det_eff_t, "Кол-во потоков\n");
fprintf(file_det_eff_t, "Эффективность\n");
fprintf(file_det_eff_t, "none\n");

FILE* file_det_eff_f = fopen("results/det_efficiency_f", "w");
fprintf(file_det_eff_f, "4\n");
fprintf(file_det_eff_f, "ReductionAlg\n");
fprintf(file_det_eff_f, "CancelAlg\n");
fprintf(file_det_eff_f, "AtomicAlg\n");
fprintf(file_det_eff_f, "Expected\n");
fprintf(file_det_eff_f, "Кол-во потоков\n");
fprintf(file_det_eff_f, "Эффективность\n");
fprintf(file_det_eff_f, "none\n");

int sz = 3*pow(10, 7);
int repeats = 20;
int max_threads = 16;
double Tlreduction, Tlatomic, Tlcancel;
for (int threads=1; threads <= max_threads; ++threads){
    int* array = (int*)malloc(sz*sizeof(int));
    int* determined_array = (int*)malloc(sz*sizeof(int));
    int num = rand()%1000000;

    for(int i=0; i<sz; i++) {
        array[i] = rand();
        determined_array = num;
    }
    int target = rand();
    int det_true = num;
    int det_false = rand();

    getExperimentDots(array, sz, target, threads, repeats, file_res, file_speedup, file_eff,
    &Tlcancel, &Tlatomic, &Tlreduction);
    getExperimentDots(determined_array, sz, det_true, threads, repeats, NULL, NULL,
    file_det_eff_t, &Tlcancel, &Tlatomic, &Tlreduction);
    getExperimentDots(determined_array, sz, det_false, threads, repeats, NULL, NULL,
    file_det_eff_f, &Tlcancel, &Tlatomic, &Tlreduction);
}

```

```
    free(array);  
    free(determined_array);  
    printf("finished %d\n", threads);  
}  
  
fclose(file_res);  
fclose(file_speedup);  
fclose(file_eff);  
fclose(file_det_eff_t);  
fclose(file_det_eff_f);  
return 0;  
}
```

Файл 1 – Замер времени, вычисление ускорения и эффективности для параллельных алгоритмов

7.2. Таблицы

Таблица 1 – Сравнение теоретических и экспериментальных данных

	Теор. значения	Редукция	pragma cancel for	atomic
Ускорение	P	1.95 – 5.08	1.91 – 5.35	1.97 – 5.10
Эффективность	1	0.31 – 0.98	0.32 – 0.96	0.31 – 0.99

Таблица 2 – Данные о времени работы параллельных алгоритмов

Кол-во потоков	ReductionAlg	CancelAlg	AtomicAlg
1	39.452815	84.642005	47.827101
2	20.210314	44.239807	24.263692
3	13.912201	30.381703	16.545963
4	10.355401	22.781205	12.664199
5	12.051916	21.334982	14.513397
6	10.312200	18.859005	12.678909
7	9.025311	16.750407	11.024499
8	8.179116	16.184282	9.941483
9	9.259987	19.894314	11.008501
10	8.327794	18.142414	9.955812
11	7.793117	16.757989	9.767699
12	7.767797	15.805316	9.368992
13	9.306598	18.260288	10.498309
14	8.857918	17.008615	10.468388
15	8.377004	16.903710	10.028481
16	7.927012	16.006398	9.617996

Таблица 3 – Данные об ускорении параллельных алгоритмов

Кол-во потоков	ReductionAlg	CancelAlg	AtomicAlg	Expected
1	1.000000	1.000000	1.000000	1
2	1.952113	1.913254	1.971139	2
3	2.835843	2.785953	2.890560	3
4	3.809878	3.715431	3.776560	4
5	3.273572	3.967287	3.295376	5
6	3.825839	4.488148	3.772178	6
7	4.371352	5.053131	4.338256	7
8	4.823604	5.229889	4.810862	8
9	4.260569	4.254583	4.344561	9
10	4.737487	4.665421	4.803938	10
11	5.062521	5.050845	4.896455	11
12	5.079023	5.355287	5.104829	12
13	4.239231	4.635305	4.555696	13
14	4.453960	4.976420	4.568717	14
15	4.709657	5.007303	4.769127	15
16	4.977009	5.288011	4.972668	16

Таблица 4 – Данные об эффективности параллельных алгоритмов

Кол-во потоков	ReductionAlg	CancelAlg	AtomicAlg	Expected
1	1.000000	1.000000	1.000000	1
2	0.976056	0.956627	0.985569	1
3	0.945281	0.928651	0.963520	1
4	0.952470	0.928858	0.944140	1
5	0.654714	0.793457	0.659075	1
6	0.637640	0.748025	0.628696	1
7	0.624479	0.721876	0.619751	1
8	0.602950	0.653736	0.601358	1
9	0.473397	0.472731	0.482729	1
10	0.473749	0.466542	0.480394	1
11	0.460229	0.459168	0.445132	1
12	0.423252	0.446274	0.425402	1
13	0.326095	0.356562	0.350438	1
14	0.318140	0.355459	0.326337	1
15	0.313977	0.333820	0.317942	1
16	0.311063	0.330501	0.310792	1